深度学习与文本处理

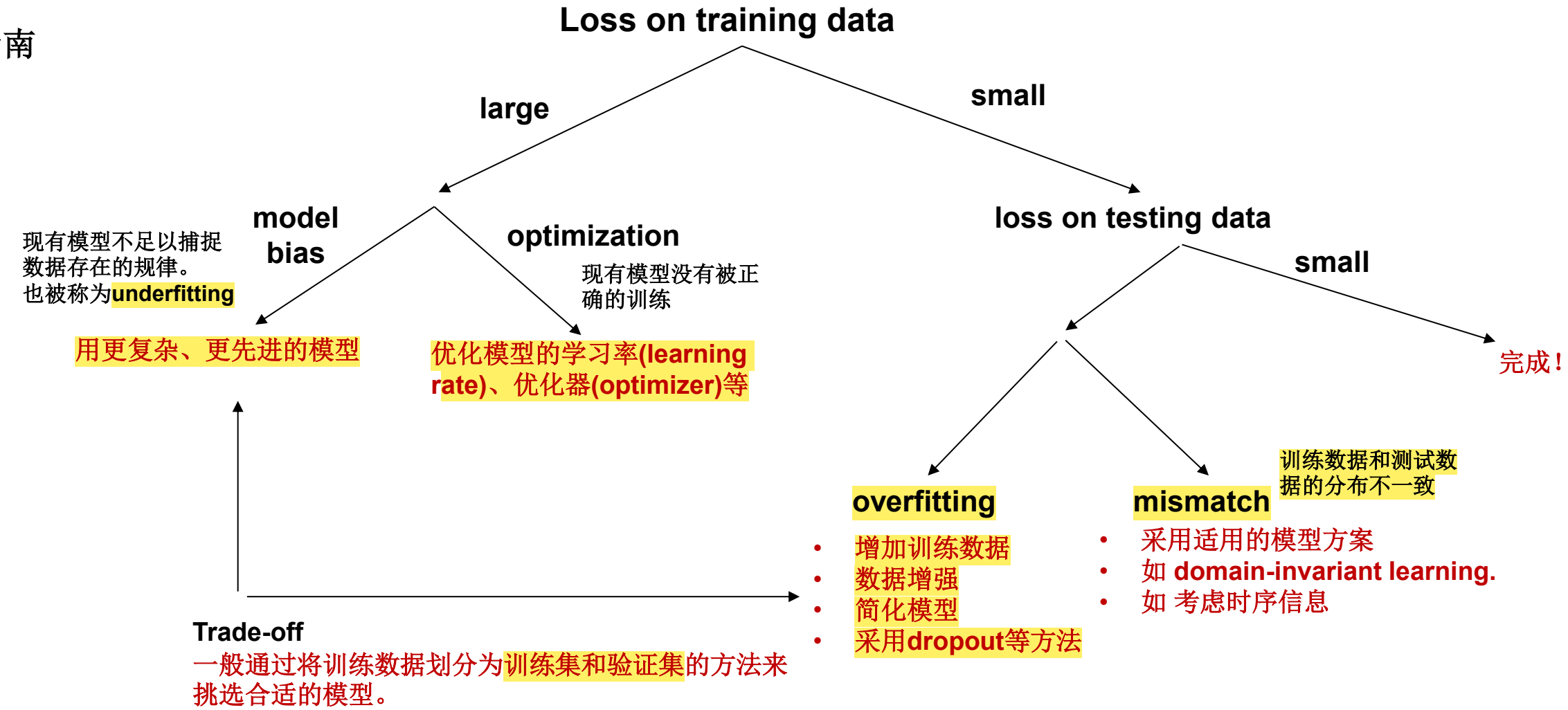
2026年3月

本节课内容

- 训练深度学习模型常见的问题和解决方法
- RNN, LSTM的原理讲解和应用

如何训练一个深度学习模型

通用指南



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

如何训练一个深度学习模型

- 一些其他常见问题
  - 训练不稳定
    - 调整学习参数;
    - 加大batch size; (注意，batch size并不是越大越好！)

batch size比较大的时候，模型就看不到属于某些小样本的规律了。

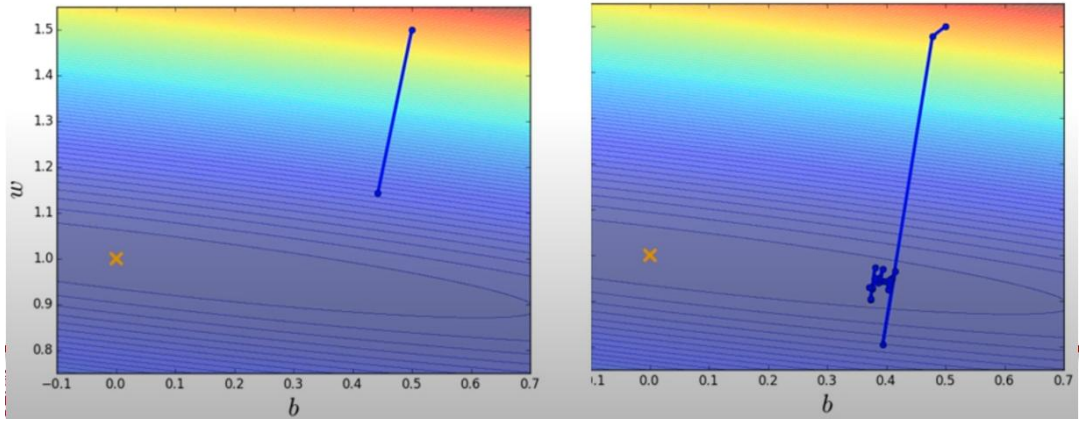
Consider 20 examples (N=20)

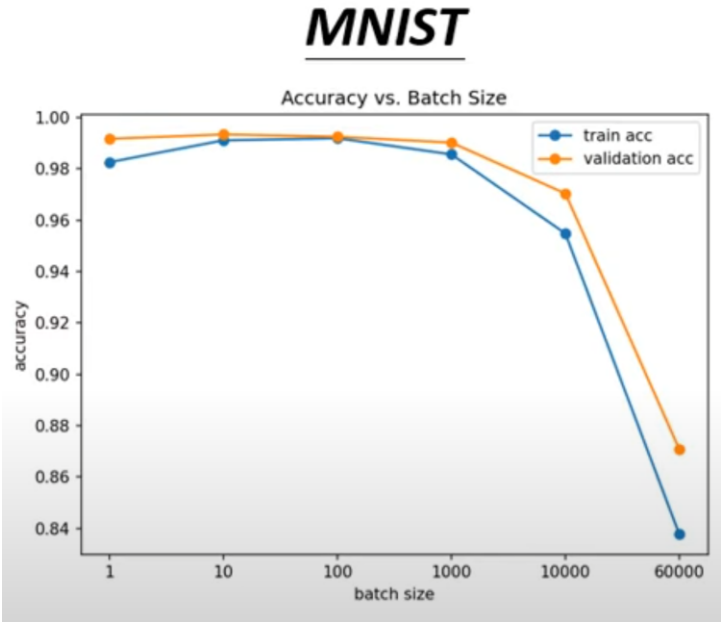
Batch size = N (Full batch)

Update after seeing all the 20 examples

Batch size = 1

Update for each example





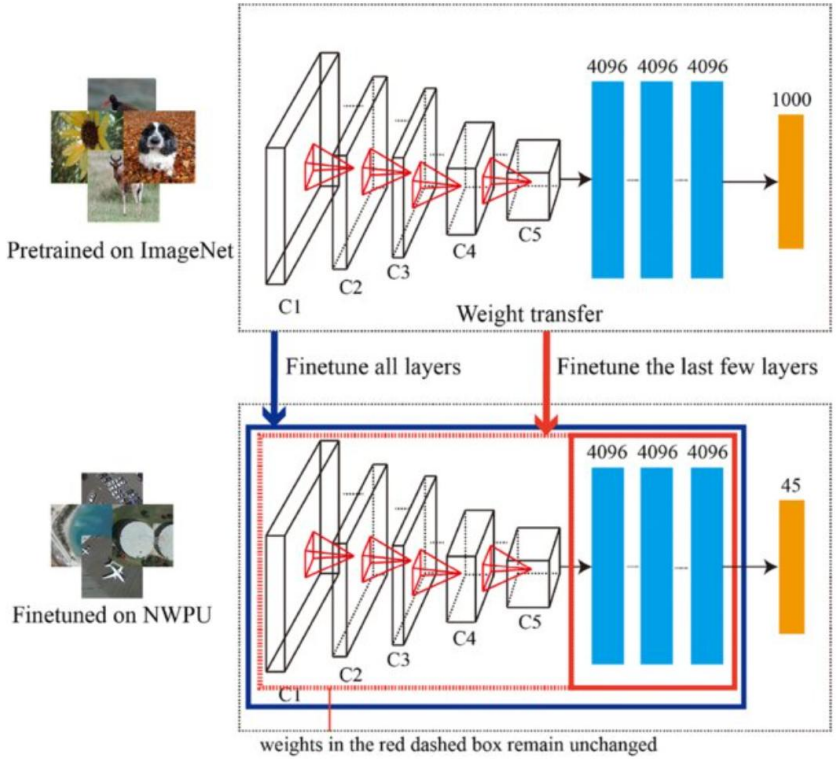
Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

如何训练一个深度学习模型

- 一些其他常见问题
  - 模型的训练目标与实际使用需求不一致
    - 整体的准确率 vs. 特定类别的区分能力　上市公司欺诈行为识别。整体样本中欺诈的公司比较少。
    - 关注整体的平均偏差 vs.重点关注与模型预测值偏差较大的样本
      - L1 Loss **vs.** L2 Loss　数据干净+追求速度→L2；数据有脏点或重尾分布→L1
  - 异常值导致loss在某个样本非常大
    - E.g., 大多数的股票波动范围在[-10%, 10%]之间，但是可能有个股票一天涨幅超过10000%
    - 需要去除异常值，或者对loss的计算结果做截断。
      - torch.nn.utils.clip_grad_value_

如何训练一个深度学习模型

- 一些其他常见问题
  - 模型参数初始化
    - 模型在创建的时候（尤其是当你自己手写一些layer的时候），一开始参数的数值分布可能比较极端。
      - 需要使用一些参数初始化的功能，如torch.nn.init.uniform_ 和torch.nn.init.xavier_uniform_
      - 基于一个已经训练好的模型，对模型结构做微调（一般只修改最后的output layer）之后，继续训练。



微调模型

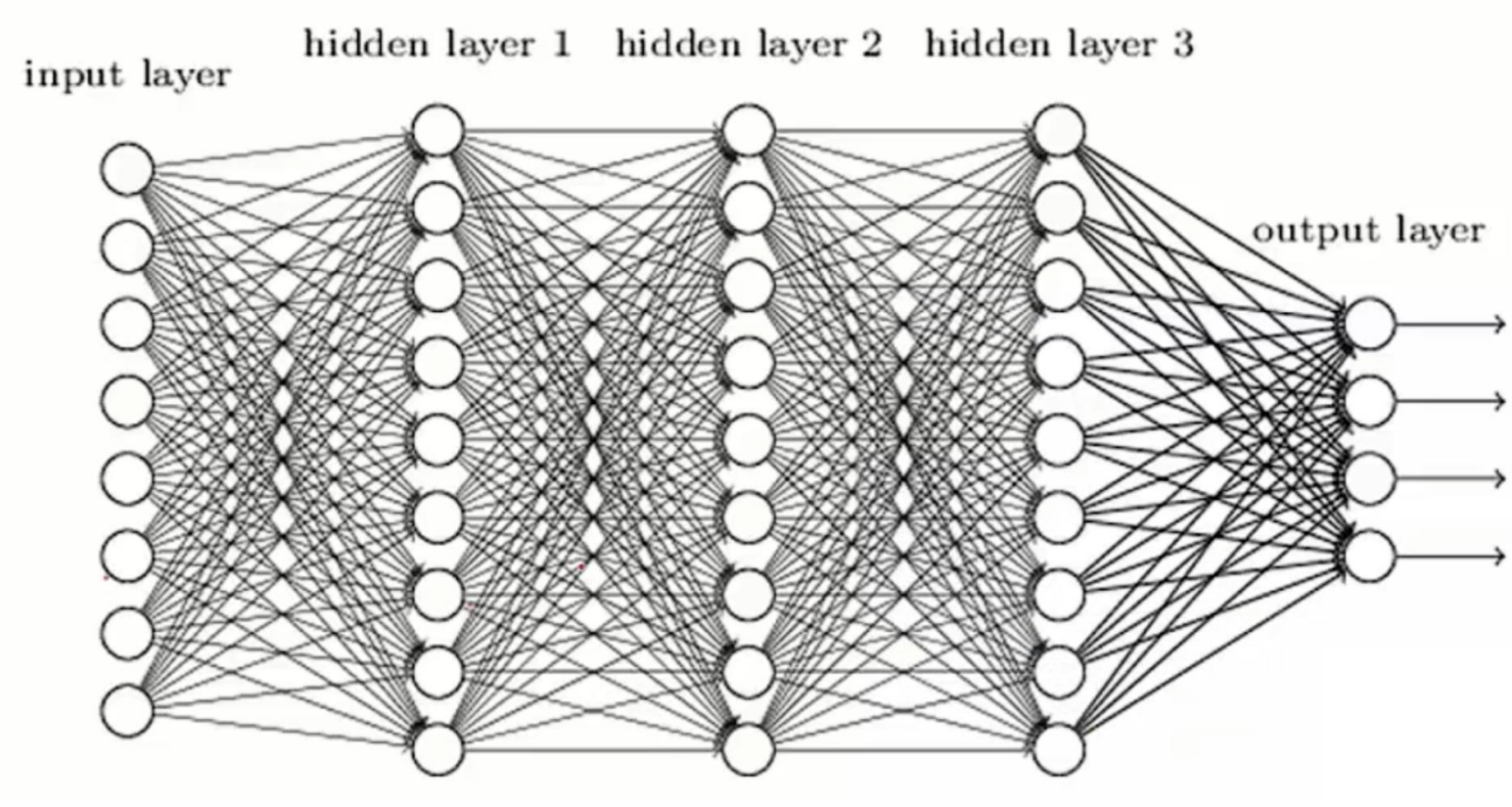
如何训练一个NLP深度学习模型

- 常见流程
  - 与传统的深度学习模型的流程类似
  - 一般需要的额外步骤：将数据结构化
    - 将非结构化的文本数据转换为结构化的向量表示（特征）。
    - 一般有两种使用方法：
      - 训练一个专门将文本数据结构化的模型（如词向量模型）。下游的模型基于转换后的数据做运算。分层的结构（hierarchical structure）。 常用的应用模型结构：hierarchical attention network
      - 文本数据结构化作为模型的一个部分。模型在训练过程中学习针对任务的最优的结构化方法。如目前经常使用的大语言模型中的embedding模块。

如何训练一个NLP深度学习模型

- 常见问题
  - 不同样本的数据形状不一致
    - 不同文本数据的长短不一。无法有效支持Batch训练。
    - 方法：（1）对文本进行补齐、截断，使得一个batch中的所有样本形状一致。（2）一次只训练一个样本
  - 文本太长，将所有数据放到模型中，可能会导致显存不足
    - 如BERT模型一般只能支持512 token长度的输入
    - 方法：对文本数据做截断；优化模型结构

从传统的神经网络到循环神经网络（RNN）



特点：

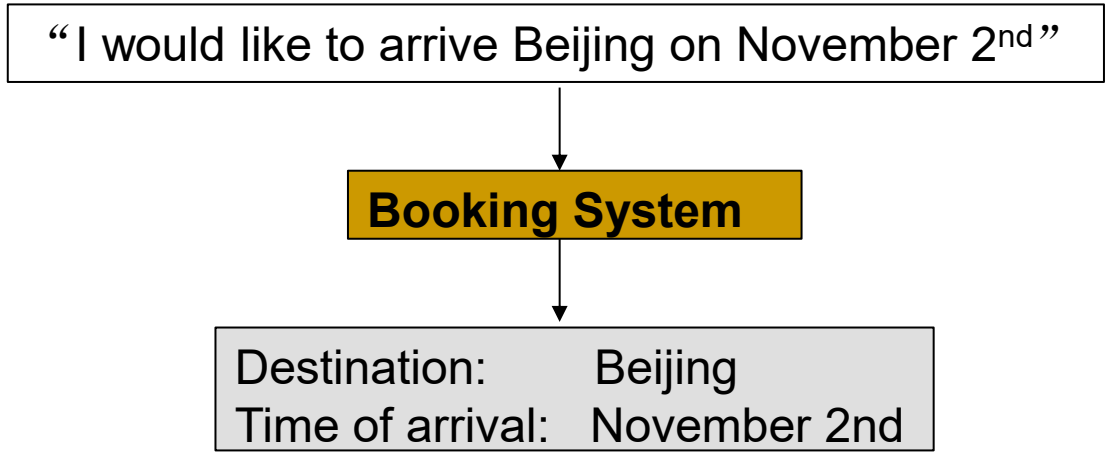
1. 输入每个样本对应的特征。
2. 输入的数据形状是固定的。
3. 每次的计算结果都是独立的

这句话的含义和上句话的含义是有关联的

文本长度通常不固定

从传统的神经网络到循环神经网络（RNN）

- 在有些任务中，数据的前后关系所包含的信息非常重要；同时，数据的形状和关键数据的位置可能不确定。
- 例：
  - 航空公司从客户发送的信息中抽取形成信息
  - 任务：分析每一个词汇，判断该词汇属于哪种信息（出发地、到达地、时间等）



从传统的神经网络到循环神经网络（RNN）

- 使用传统的feedforward network来完成任务？
  - 模型基于每个词的信息做判断
  - 问题：对于传统的neutral network, 输入的数据一样，输出的结果就是一样的。
  - “I would like to arrive Beijing on November 2nd”
  - “I would like to leave Beijing on November 2nd”
  - 神经网络需要记忆力。

Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

从传统的神经网络到循环神经网络（RNN）

RNN

CLASS torch.nn.RNN(*input_size, hidden_size, num_layers=1, nonlinearity='tanh', bias=True, batch_first=False, dropout=0.0, bidirectional=False, device=None, dtype=None*) [SOURCE]

Apply a multi-layer Elman RNN with $\tanh$ or ReLU non-linearity to an input sequence. For each element in the input sequence, each layer computes the following function:

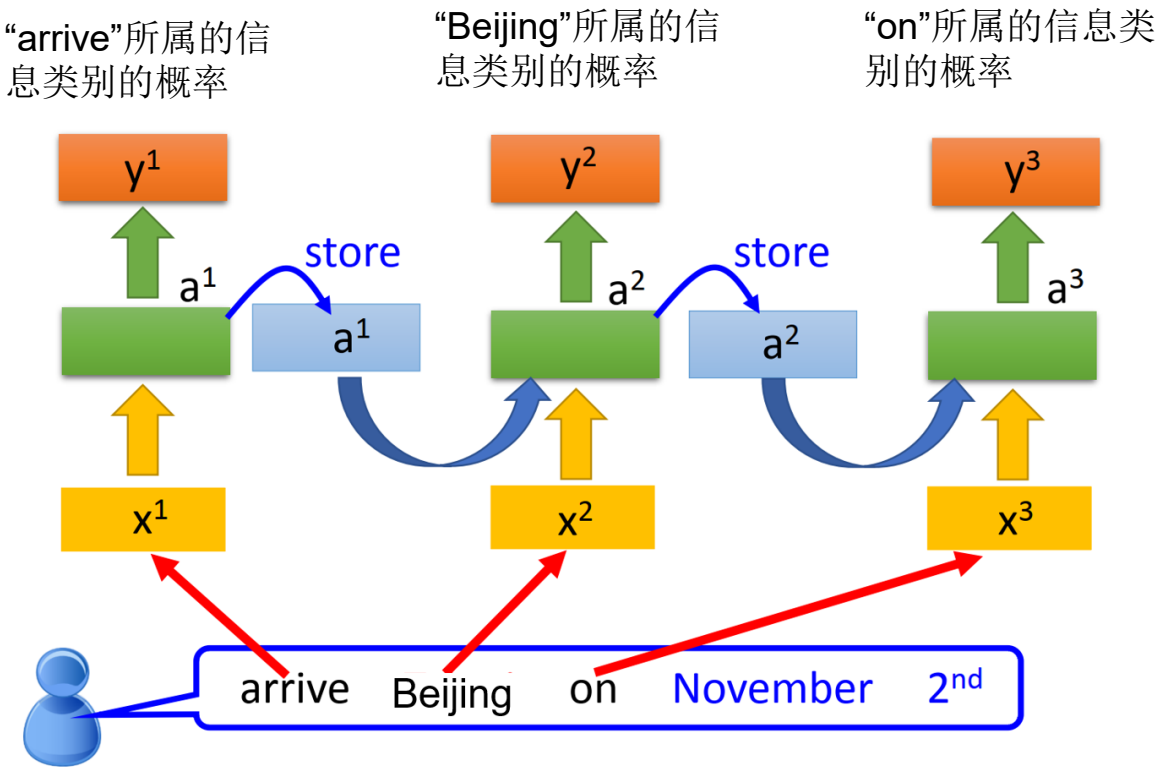
$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$

where h_t is the hidden state at time *t*, x_t is the input at time *t*, and $h_{(t-1)}$ is the hidden state of the previous layer at time *t-1* or the initial hidden state at time *0*. If `nonlinearity` is `'relu'`, then ReLU is used instead of $\tanh$.

```python
# Efficient implementation equivalent to the following with bidirectional=False
def forward(x, hx=None):
    if batch_first:
        x = x.transpose(0, 1)
    seq_len, batch_size, _ = x.size()
    if hx is None:
        hx = torch.zeros(num_layers, batch_size, hidden_size)
    h_t_minus_1 = hx
    h_t = hx
    output = []
    for t in range(seq_len):
        for layer in range(num_layers):
            h_t[layer] = torch.tanh(
                x[t] @ weight_ih[layer].T
                + bias_ih[layer]
                + h_t_minus_1[layer] @ weight_hh[layer].T
                + bias_hh[layer]
            )
        output.append(h_t[-1])
        h_t_minus_1 = h_t
    output = torch.stack(output)
    if batch_first:
        output = output.transpose(0, 1)
    return output, h_t
```

循环神经网络（RNN）

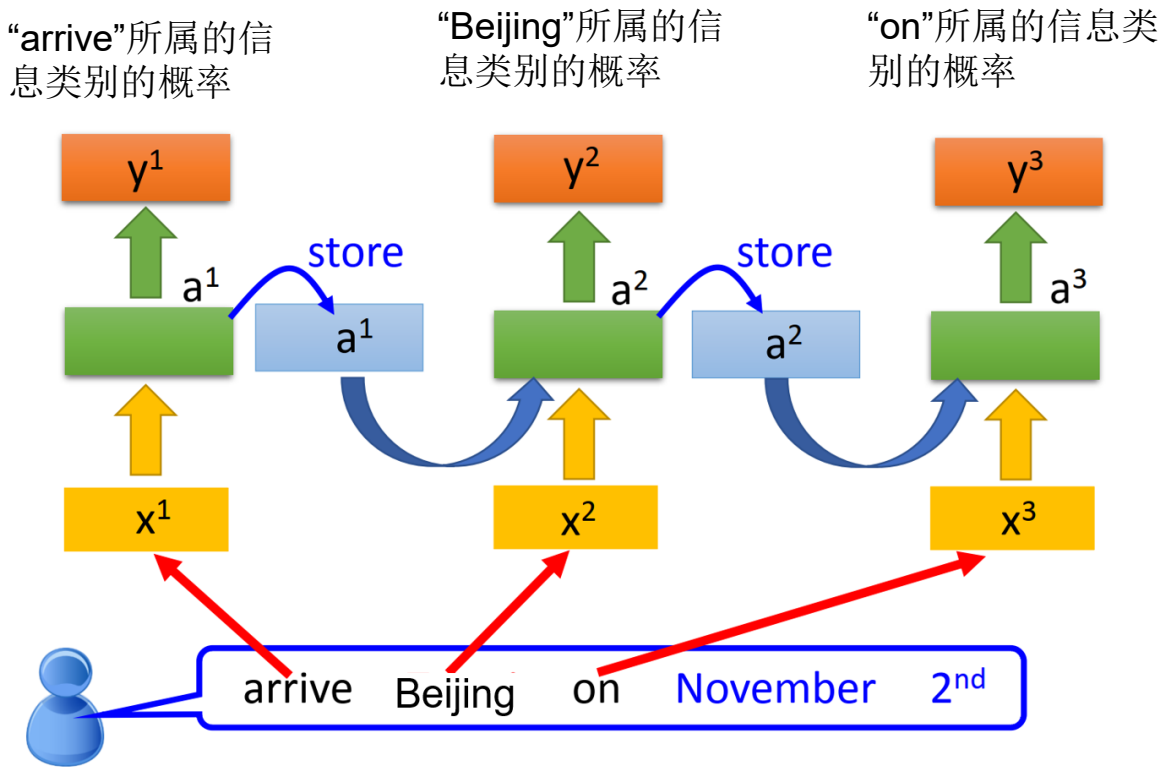
- 将一个网络循环使用。



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

循环神经网络（RNN）

- 将一个网络循环使用。

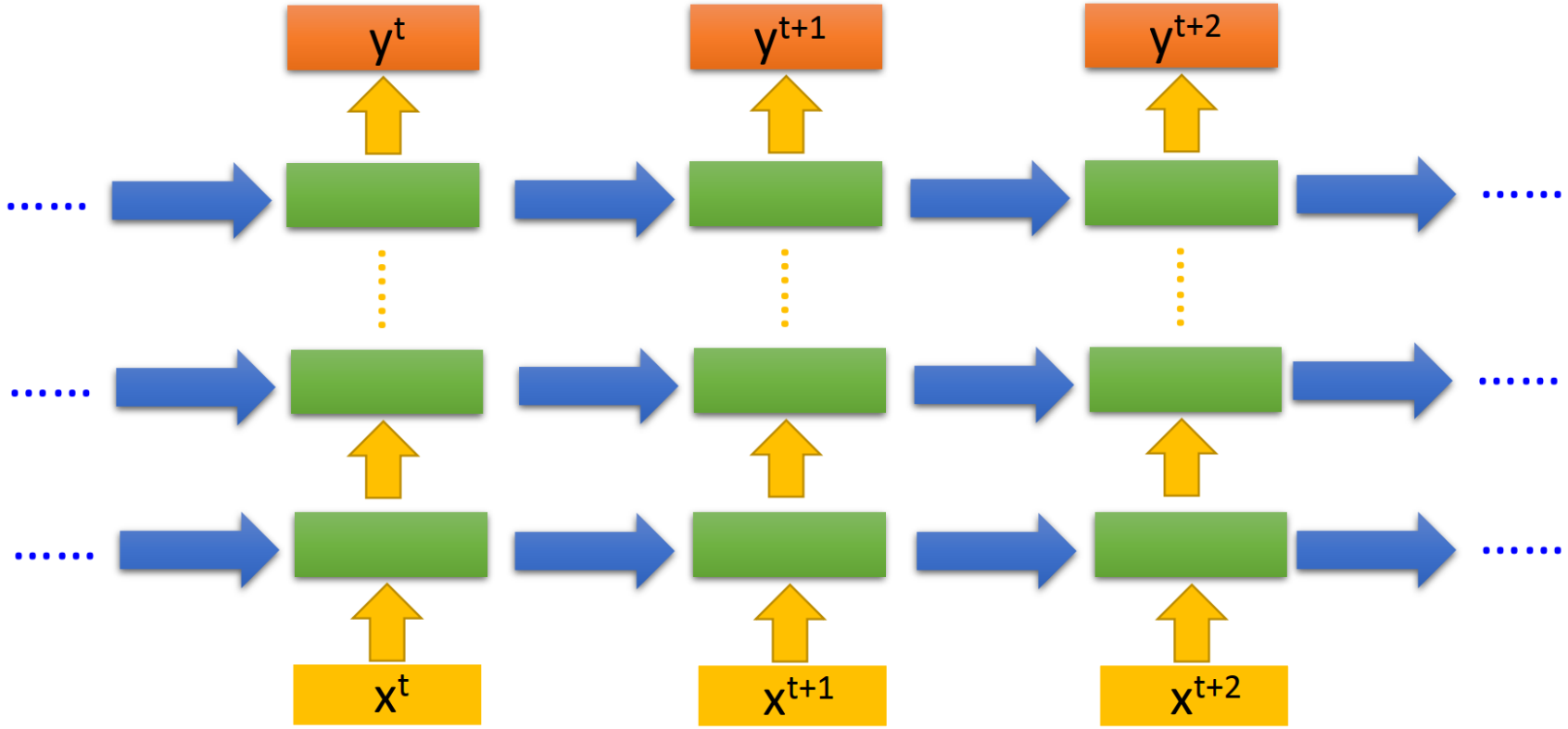


如果将“arrive”改为“leave”,保存在a_1中的数据也会发生改变，从而改变y_2的预测结果。

Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

循环神经网络（RNN）

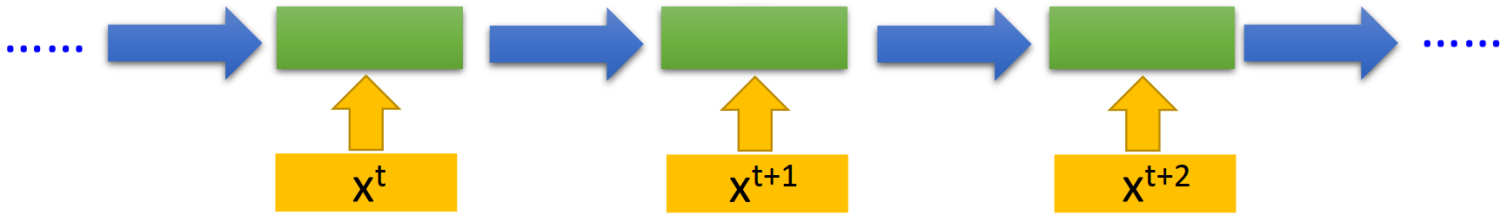
- 可以通过增加层数得到更复杂的模型



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

循环神经网络（RNN）

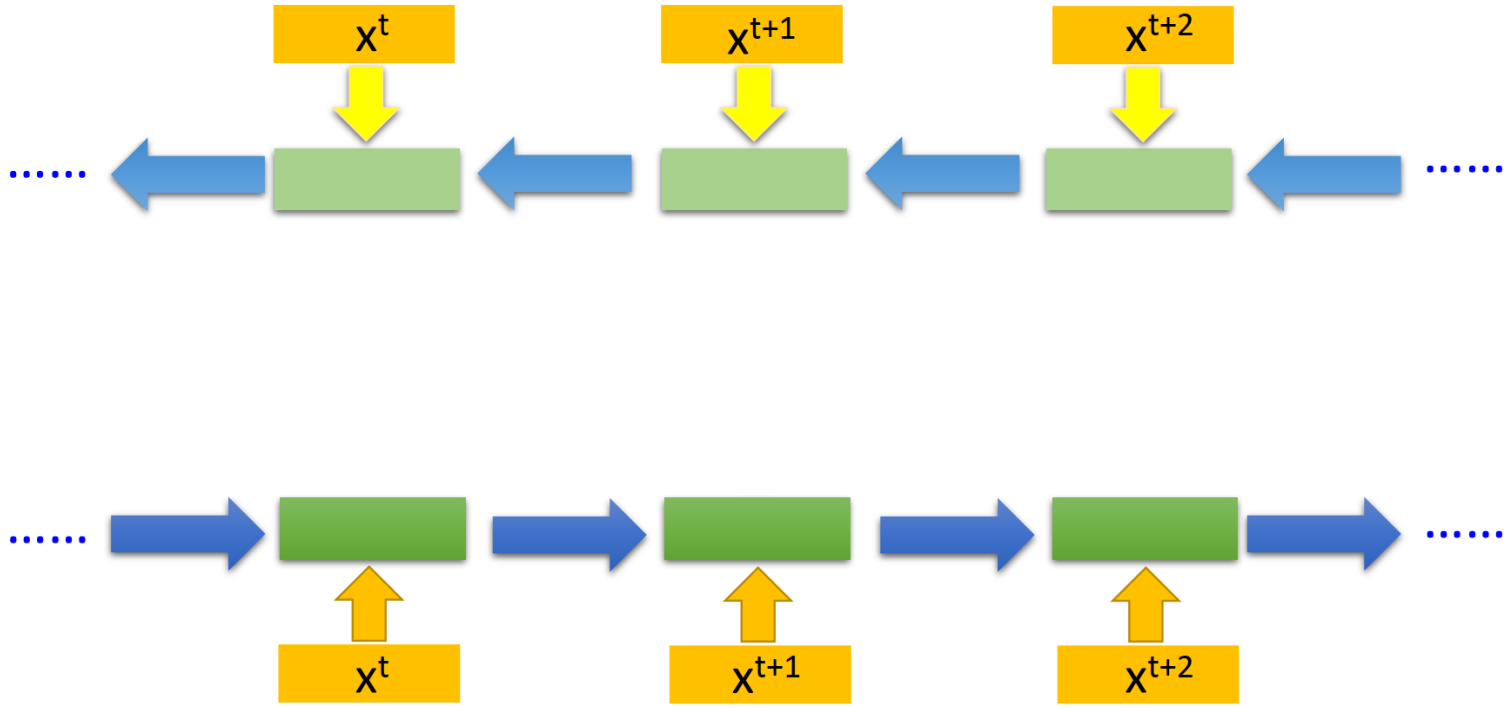
- 除了考虑前文的信息，也可以考虑后文的信息。
- 双向循环神经网络（英语： Bidirectional recurrent neural networks，BRNN）将两个方向相反的隐藏层连接到同一个输出。



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

循环神经网络（RNN）

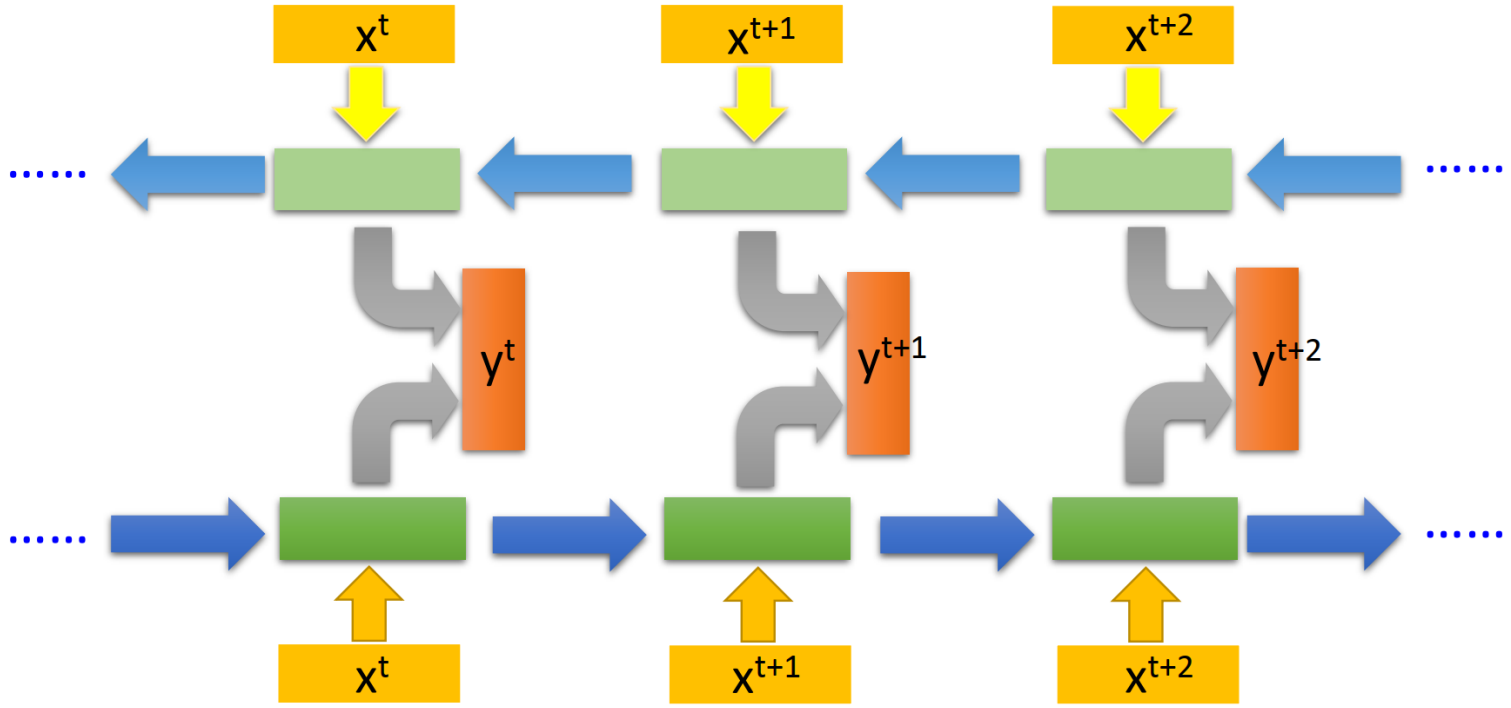
- 除了考虑前文的信息，也可以考虑后文的信息。
- 双向循环神经网络（英语： Bidirectional recurrent neural networks，BRNN）将两个方向相反的隐藏层连接到同一个输出。



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

循环神经网络（RNN）

- 除了考虑前文的信息，也可以考虑后文的信息。
- 双向循环神经网络（英语： Bidirectional recurrent neural networks，BRNN）将两个方向相反的隐藏层连接到同一个输出。

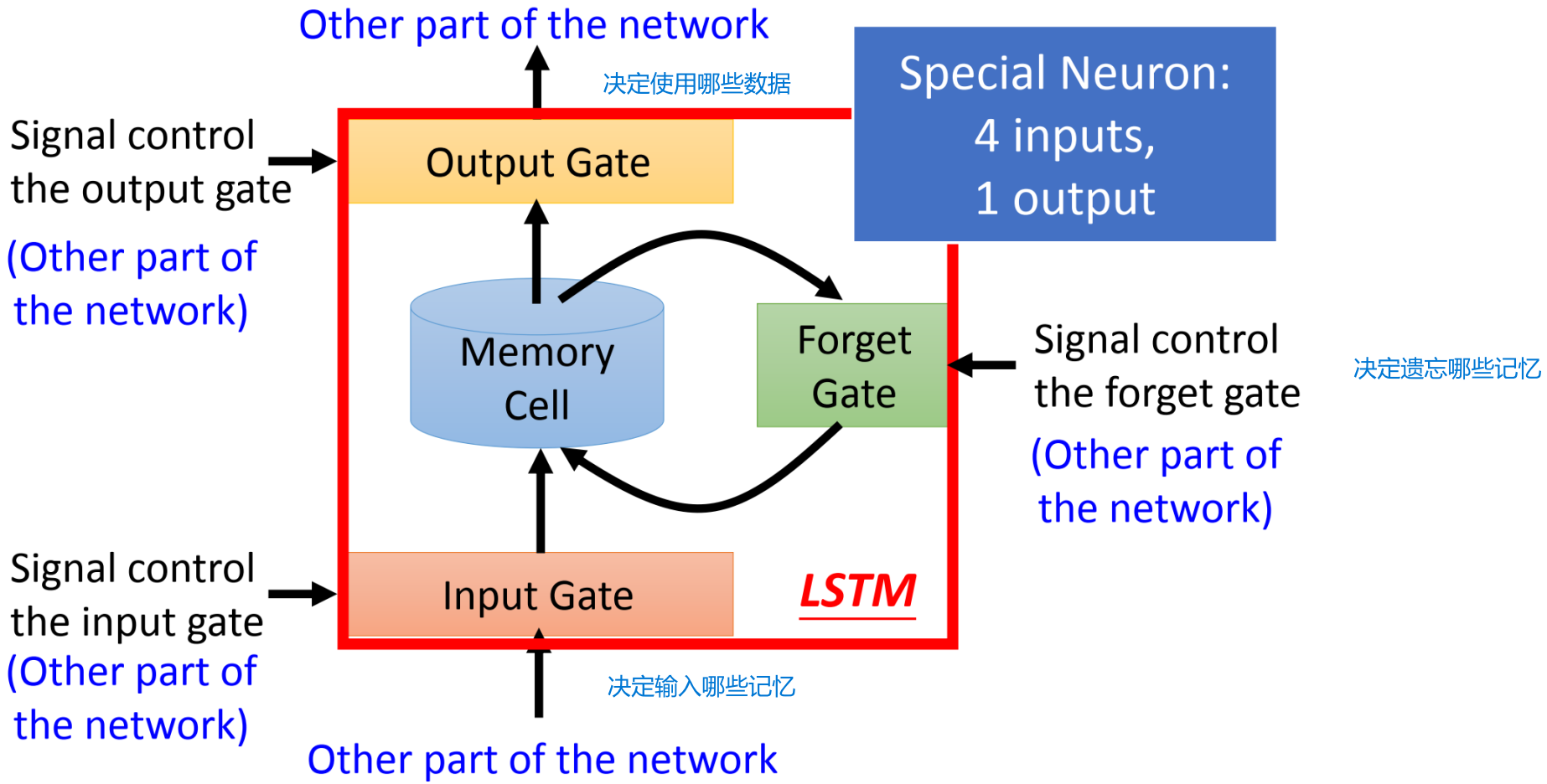


Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

Long Short-term Memory (LSTM)

RNN的问题是它对记忆的输入、记忆的遗忘、记忆的使用都没有选择性。

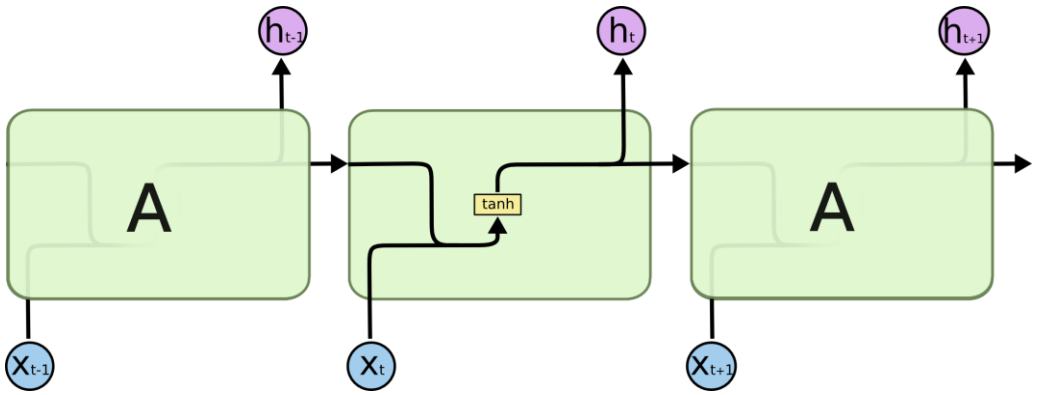
- 并不是所有前文的信息都是需要被保存的、对当前的预测有用的。



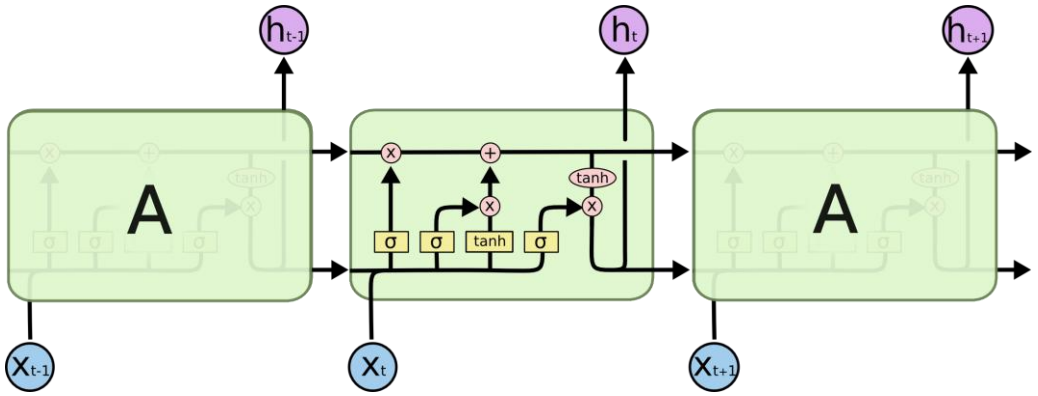
Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

Long Short-term Memory (LSTM)

RNN



LSTM

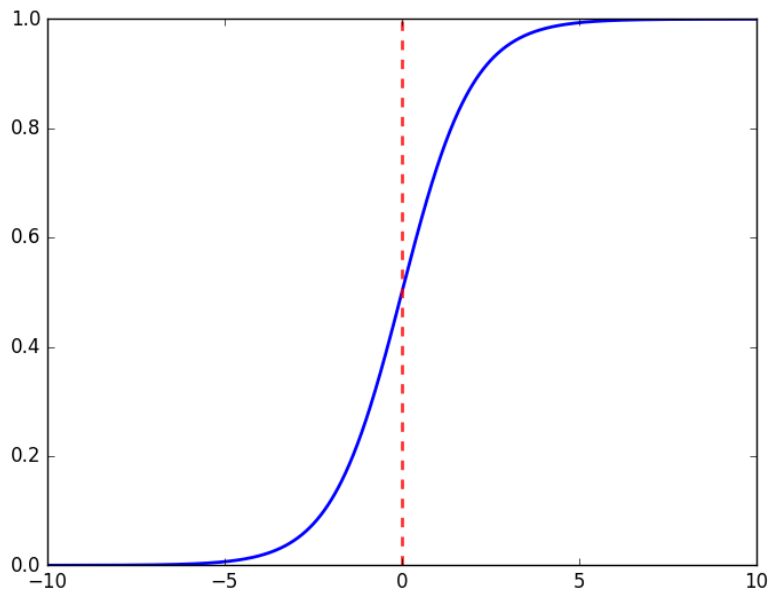


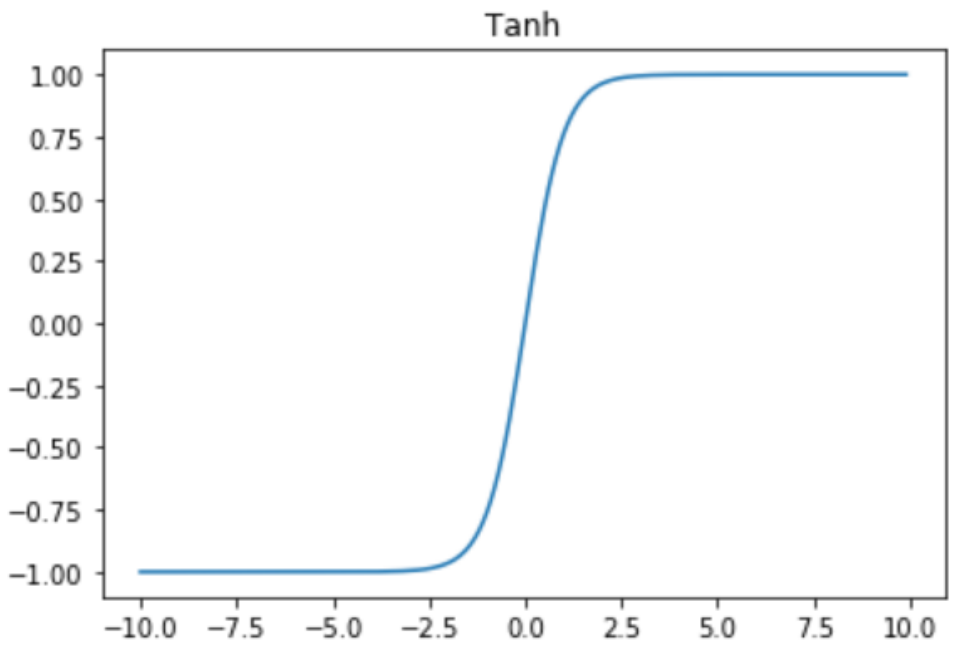
Long Short-term Memory (LSTM)

$$\text{Sigmoid}(x) = \sigma(x) = \frac{1}{1+\exp(-x)}$$

用于控制门，通过给sigmoid一个x，然后乘某个值。

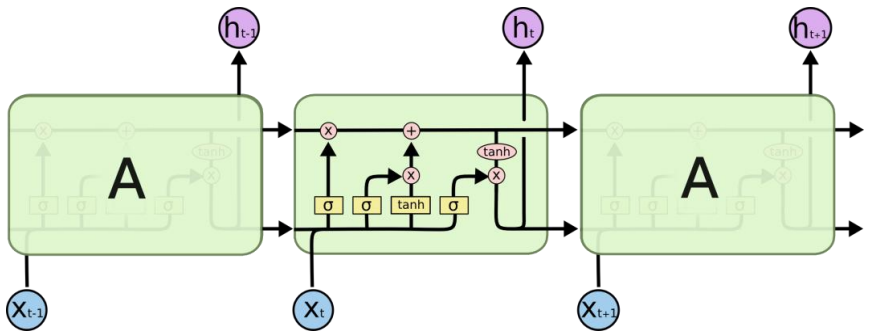
$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

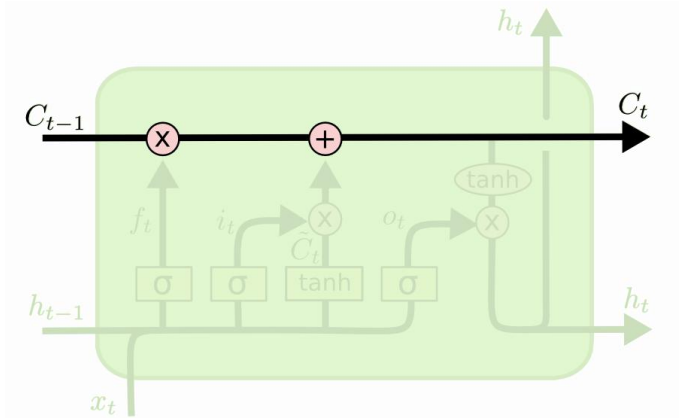




激活函数 Tanh 作用在于帮助调节流经网络的值，使得数值始终限制在 -1 和 1 之间。

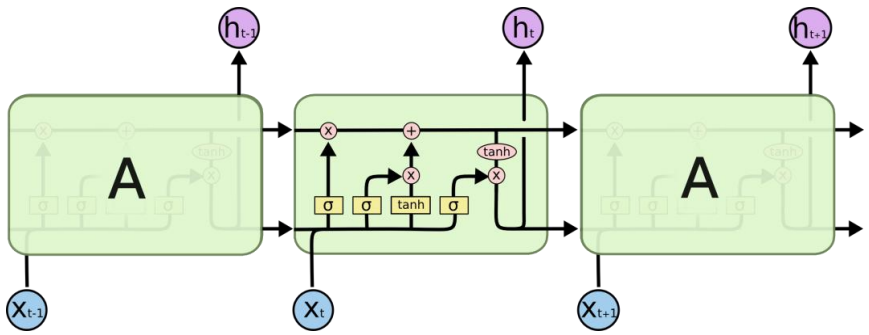
Long Short-term Memory (LSTM)





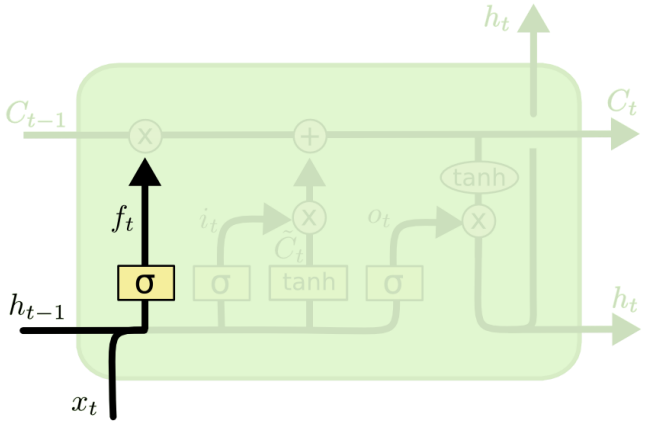
LSTM的关键就是细胞状态，水平线在图上方贯穿运行。细胞状态类似于传送带。直接在整个链上运行，只有一些少量的线性交互。信息在上面流传保持不变会很容易。

Long Short-term Memory (LSTM)

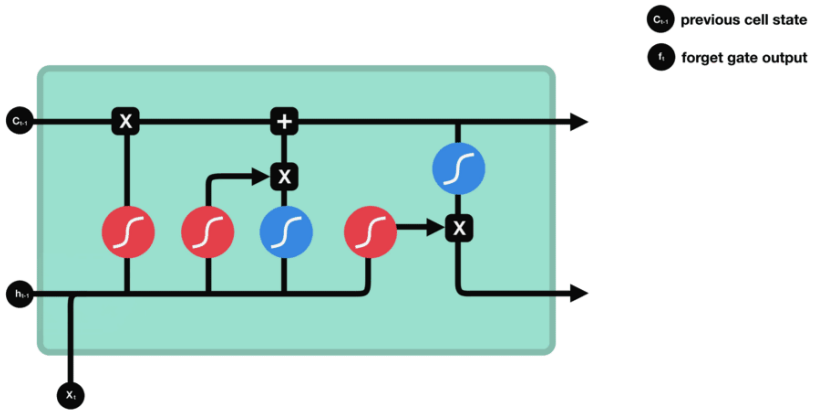


忘记门:

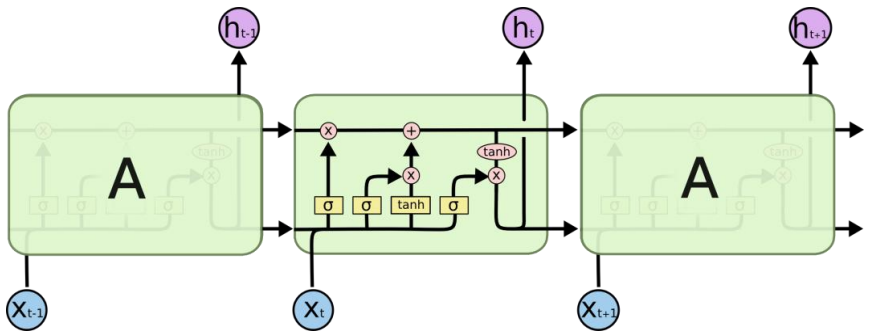
决定我们会从细胞状态中丢弃什么信息。该忘记门会读取上一个输出h_{t-1}和当前输入x_t，做一个Sigmoid 的非线性映射，然后输出一个向量f_t（该向量每一个维度的值都在0到1之间，1表示完全保留，0表示完全舍弃，相当于记住了重要的，忘记了无关紧要的），最后与细胞状态c_{t-1}相乘。



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \ + \ b_f\right)$$

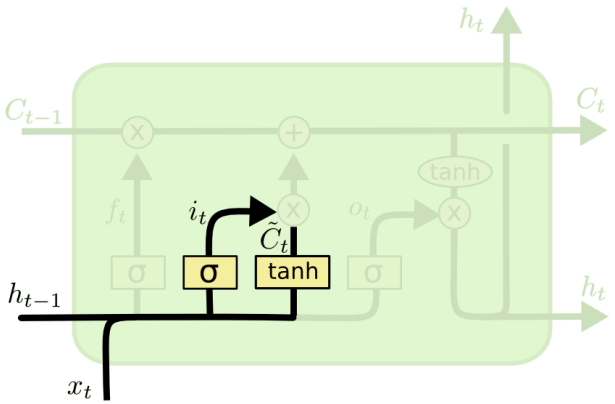


Long Short-term Memory (LSTM)



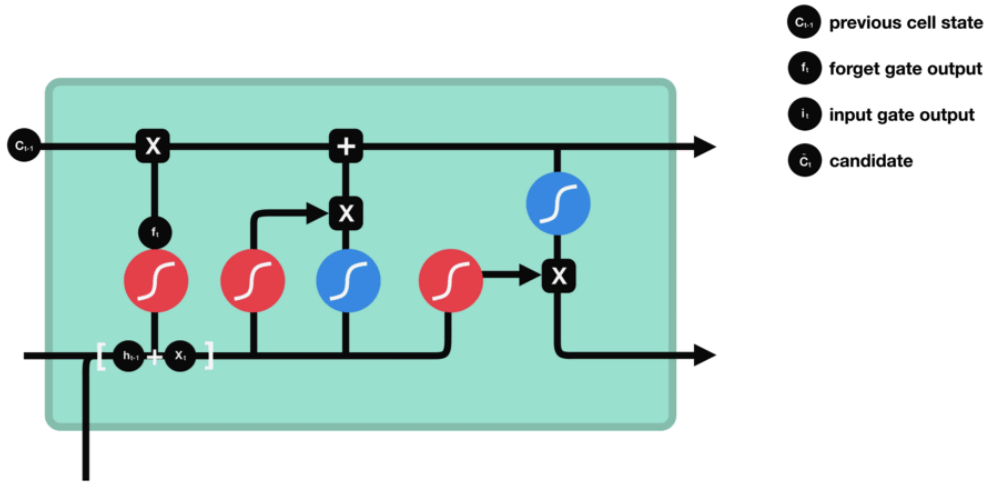
输入门:

确定什么样的新信息被存放在细胞状态中。
第一，sigmoid层称“输入门层”决定什么值我们将要更新
第二，一个tanh层创建一个新的候选值向量$\tilde{C}_t$，会被加入到状态中。

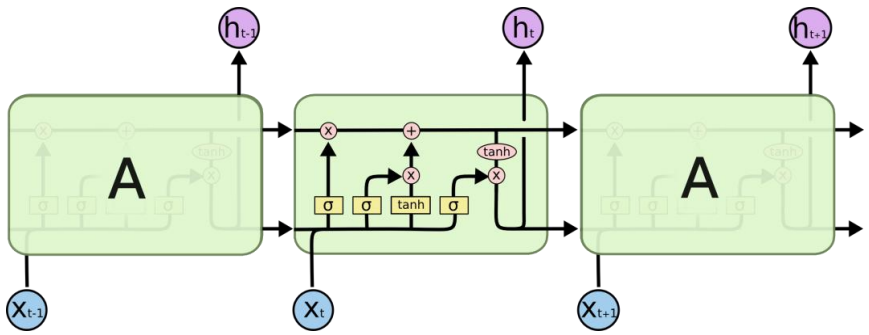


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

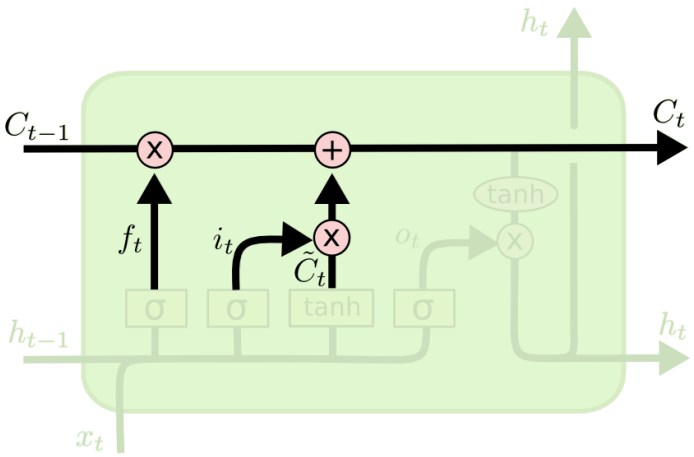


Long Short-term Memory (LSTM)

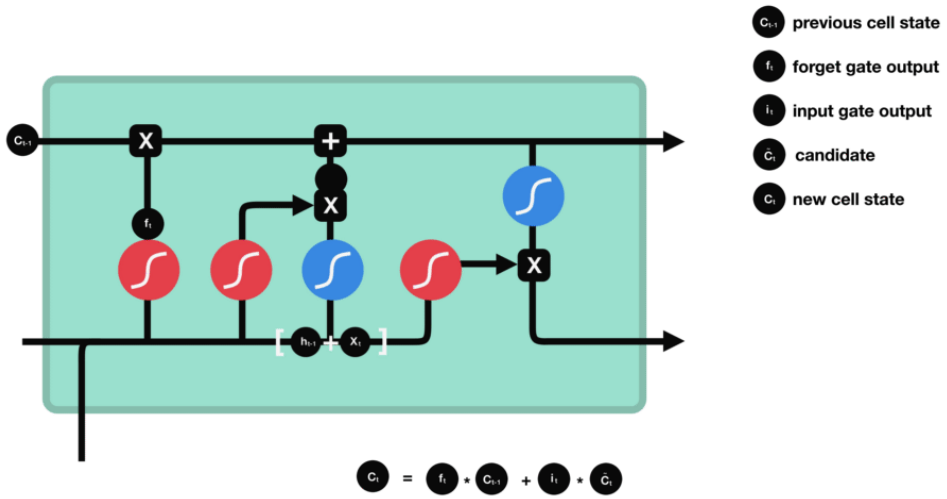


更新旧细胞状态:

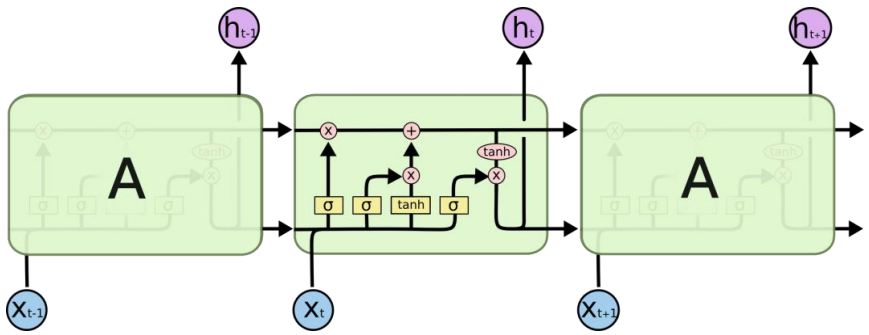
我们把旧状态与f_t相乘，丢弃掉我们确定需要丢弃的信息，接着加上$i_t * \tilde{C}_t$这就是新的候选值，根据我们决定更新每个状态的程度进行变化。



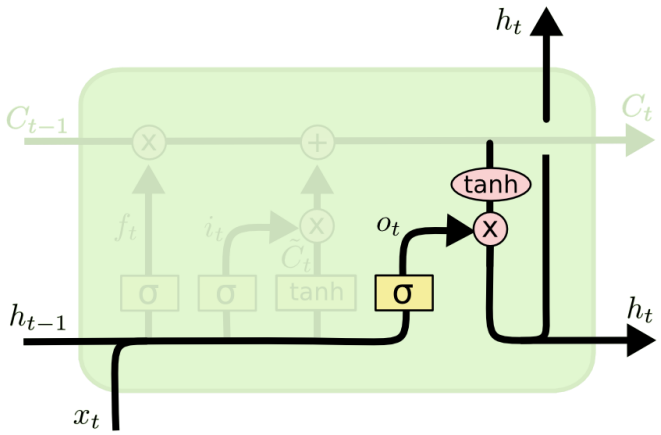
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



Long Short-term Memory (LSTM)

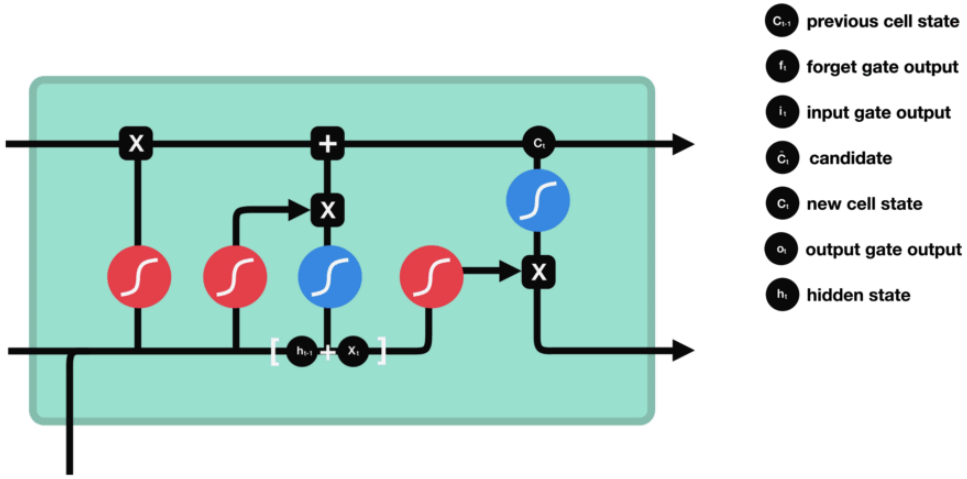


输出门:
确定最后的输出值。
首先，我们运行一个sigmoid层来确定细胞状态的哪个部分将输出出去。
接着，我们把细胞状态通过tanh进行处理（得到一个在-1到1之间的值）并将它和sigmoid门的输出相乘，最终我们仅仅会输出我们确定输出的那部分。



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$



Long Short-term Memory (LSTM)

- 好消息！ Pytorch提供实现好了的LSTM layer

LSTM

CLASS torch.nn.LSTM(*input_size, hidden_size, num_layers=1, bias=True, batch_first=False, dropout=0.0, bidirectional=False, proj_size=0, device=None, dtype=None*) [SOURCE]

Apply a multi-layer long short-term memory (LSTM) RNN to an input sequence. For each element in the input sequence, each layer computes the following function:

$$
\begin{aligned}
i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\
o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

where h_t is the hidden state at time *t*, c_t is the cell state at time *t*, x_t is the input at time *t*, h_{t-1} is the hidden state of the layer at time *t-1* or the initial hidden state at time *0*, and i_t, f_t, g_t, o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer LSTM, the input $x_t^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h_t^{(l-1)}$ of the previous layer multiplied by dropout $\delta_t^{(l-1)}$ where each $\delta_t^{(l-1)}$ is a Bernoulli random variable which is 0 with probability `dropout`.

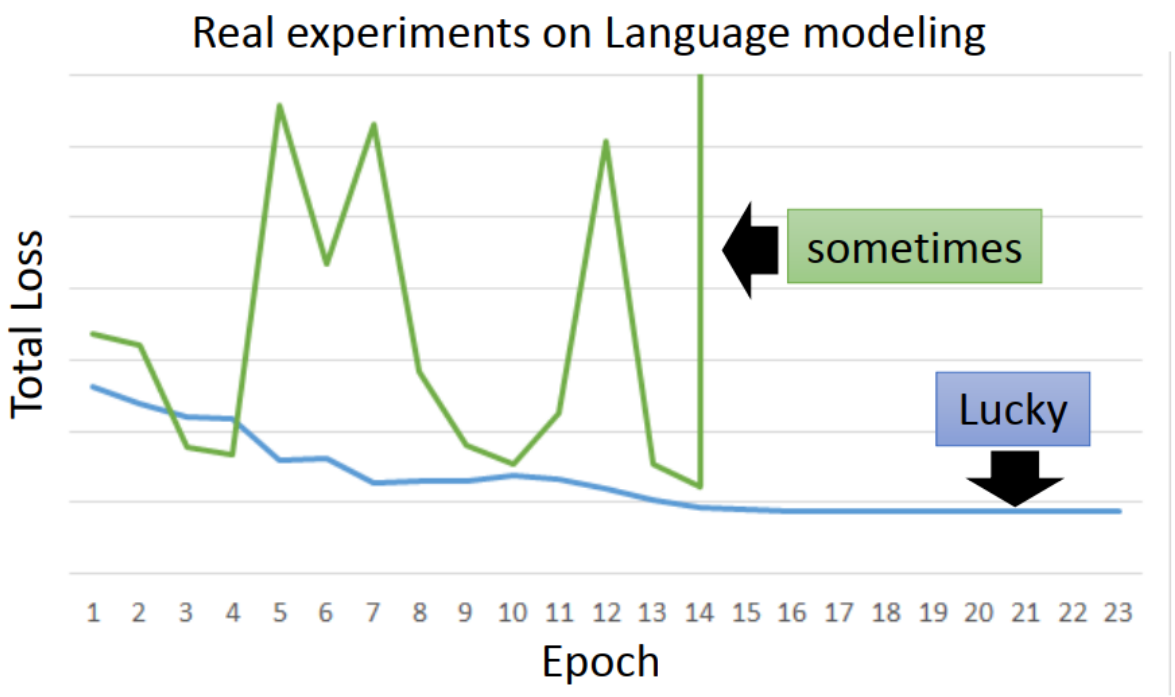
If `proj_size > 0` is specified, LSTM with projections will be used. This changes the LSTM cell in the following way. First, the dimension of h_t will be changed from `hidden_size` to `proj_size` (dimensions of W_{hi} will be changed accordingly). Second, the output hidden state of each layer will be multiplied by a learnable projection matrix: $h_t = W_{hr}h_t$. Note that as a consequence of this, the output of LSTM network will be of different shape as well. See Inputs/Outputs sections below for exact dimensions of all variables. You can find more details in https://arxiv.org/abs/1402.1128.

Examples:

```
>>> rnn = nn.LSTM(10, 20, 2)
>>> input = torch.randn(5, 3, 10)
>>> h0 = torch.randn(2, 3, 20)
>>> c0 = torch.randn(2, 3, 20)
>>> output, (hn, cn) = rnn(input, (h0, c0))
```

其他细节……

- 然而，RNN系列的模型可能很难训练好



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

其他细节……

- 然而，RNN系列的模型可能很难训练好

梯度爆炸（梯度消失）

$w = 1 \Rightarrow y^{1000} = 1$

$w = 1.01 \Rightarrow y^{1000} \approx 20000$

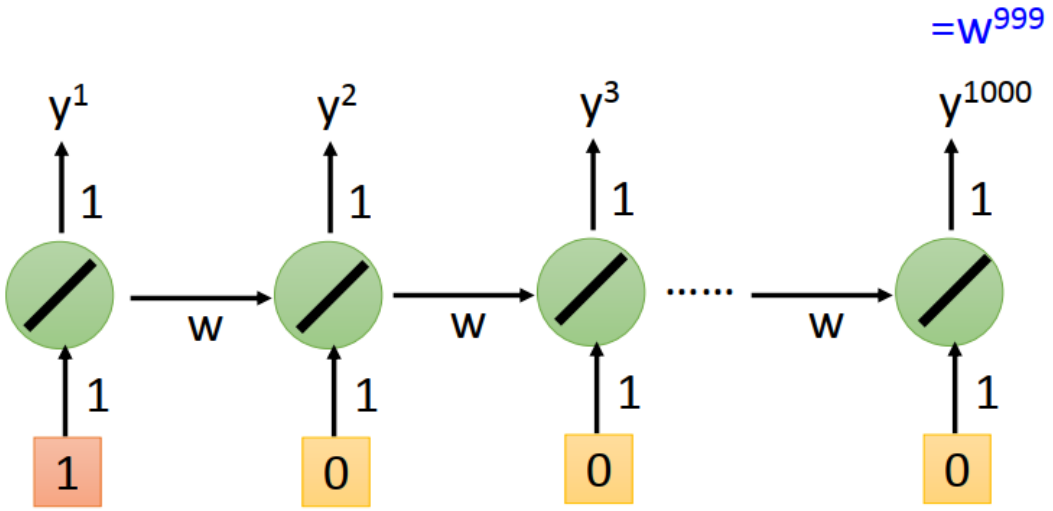
Large $\partial L/\partial w$ → Small Learning rate?

$w = 0.99 \Rightarrow y^{1000} \approx 0$

$w = 0.01 \Rightarrow y^{1000} \approx 0$

small $\partial L/\partial w$ → Large Learning rate?

Toy Example

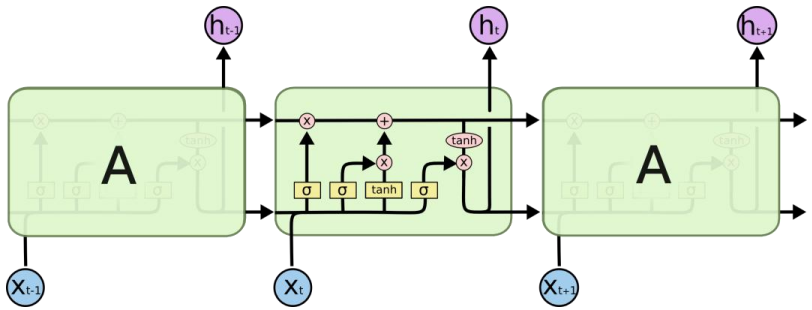


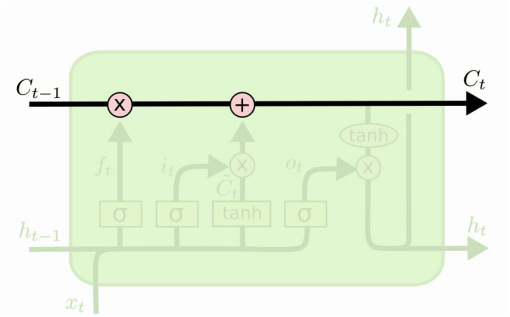
Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

其他细节……

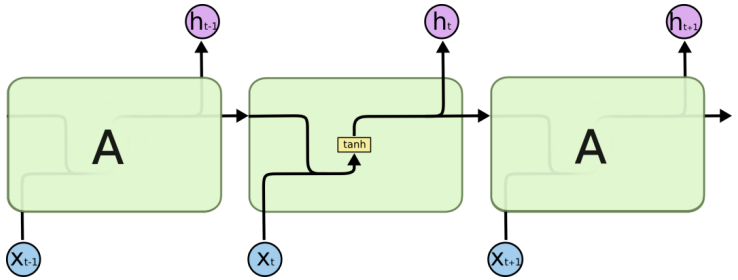
- LSTM可以解决梯度消失问题（Vanishing gradient problem）
  - RNN每次都会全部更新”memory”里面的数据；
  - LSTM每次将旧memory + 新memory通过线性操作合并起来。
    - 除非忘记门（forget gate）完全关闭。（最开始的LSTM版本没有forget gate）
- 解决梯度消失问题的其他方法
  - Clockwise RNN
  - Structurally Constrained Recurrent Network (SCRN)

LSTM

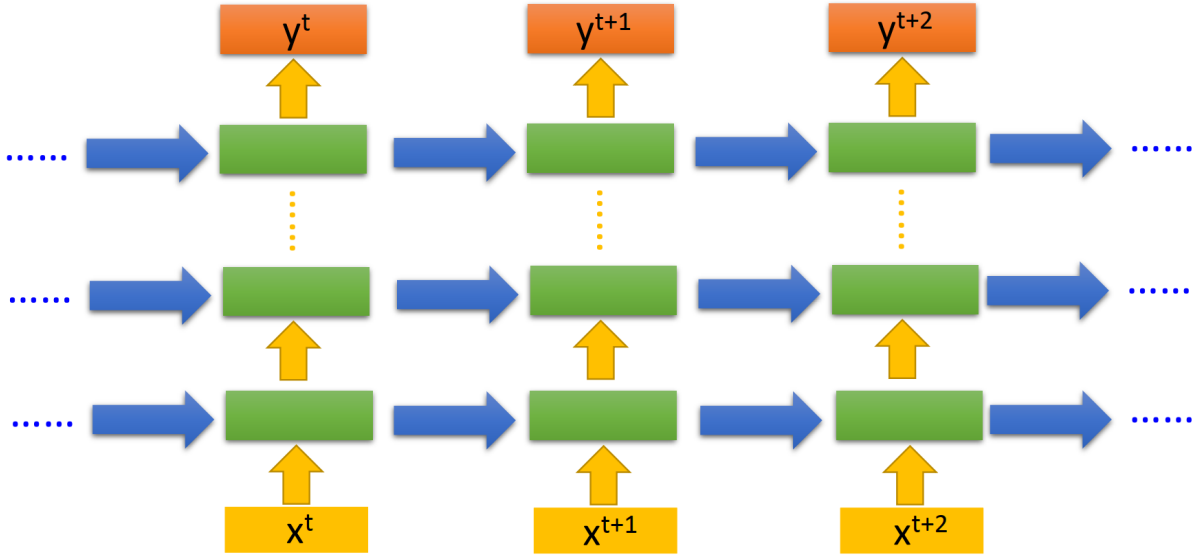




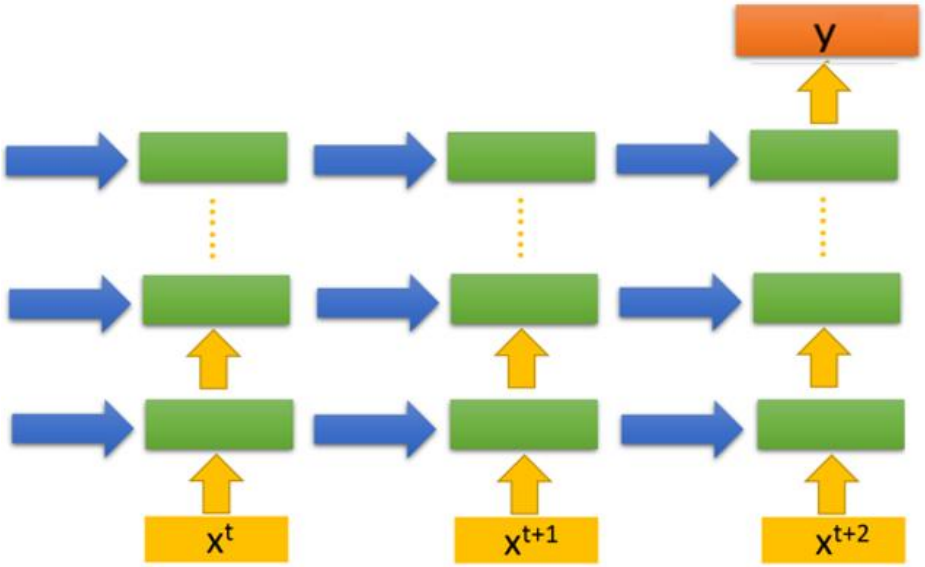
RNN



应用



Many to many

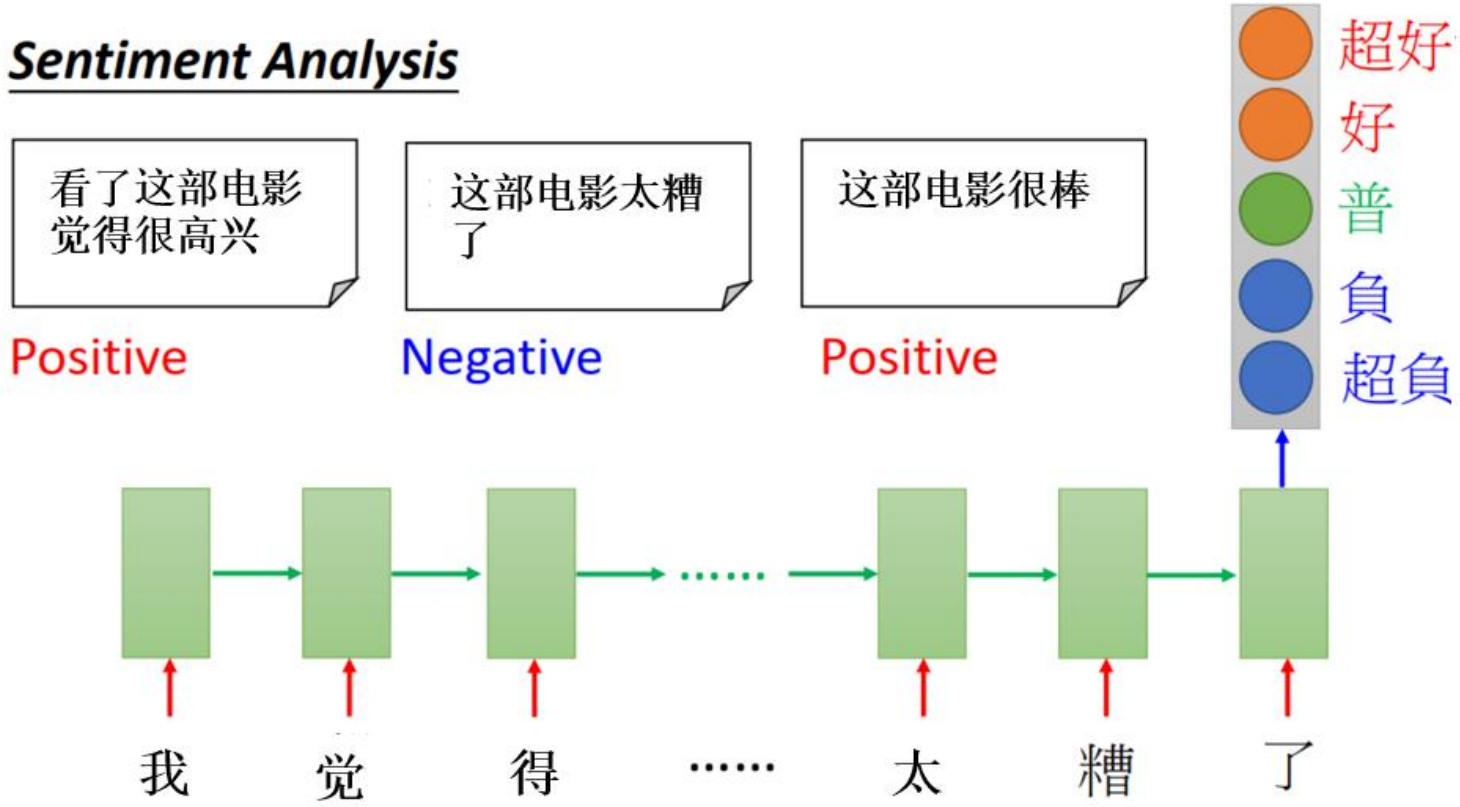


Many to one

Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

应用—Many to one

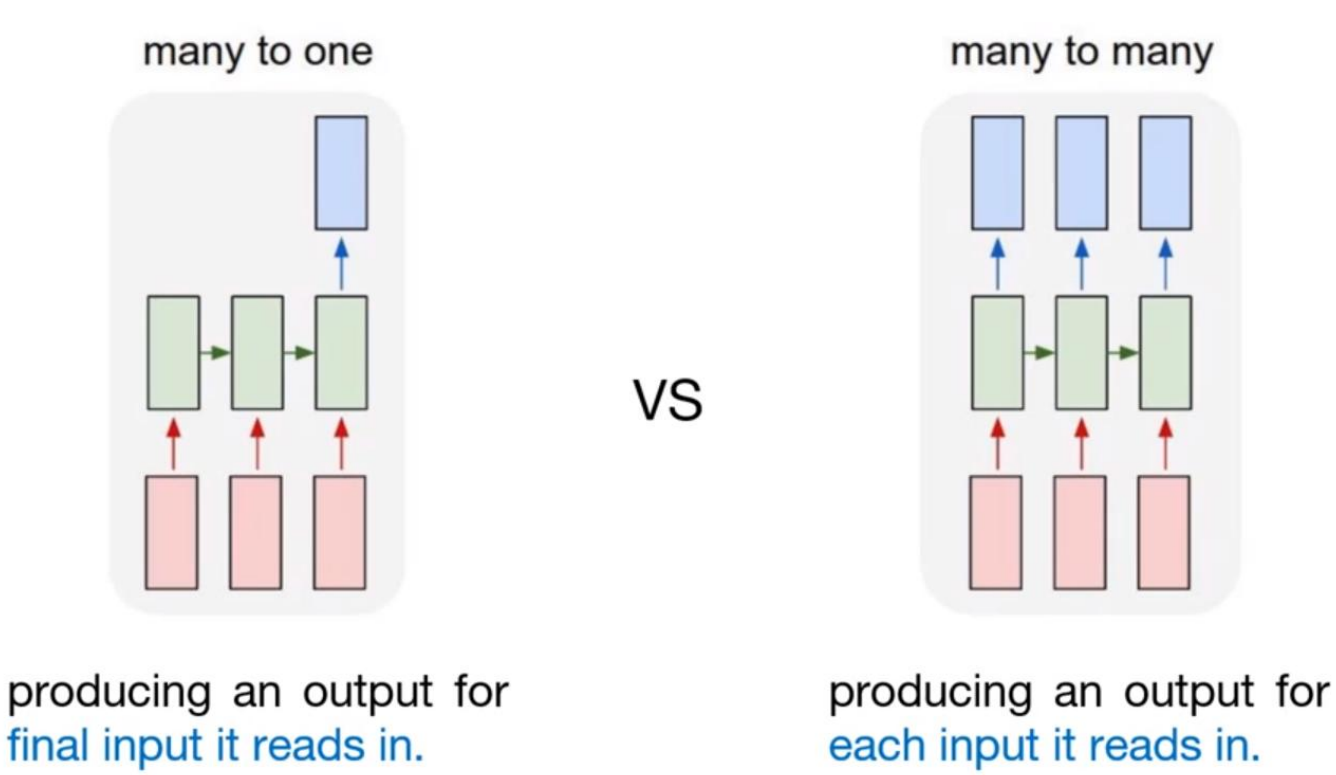
- 输入是一个vector的序列，输出是单个vector.
  - 比如文本情感识别

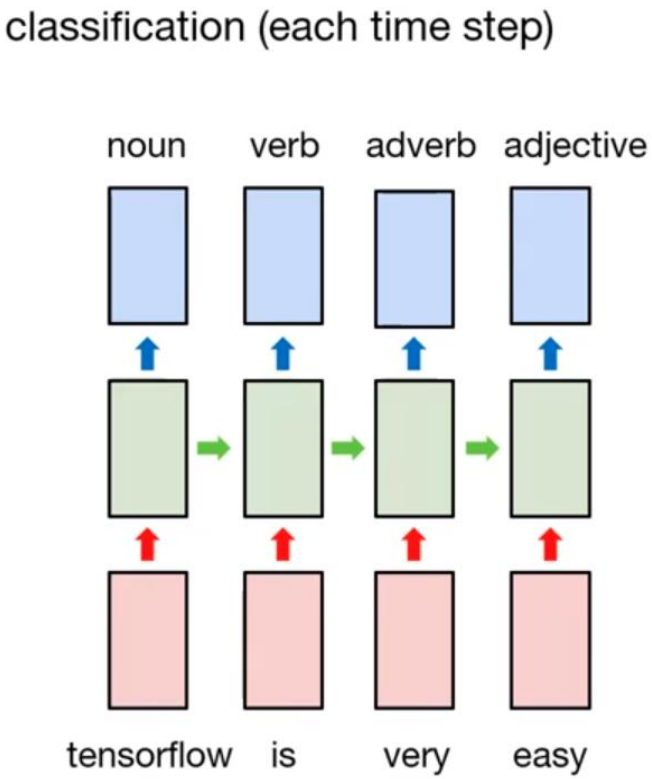


Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

应用—Many to many

- 输入是一个vector的序列，输出是一个vector的序列

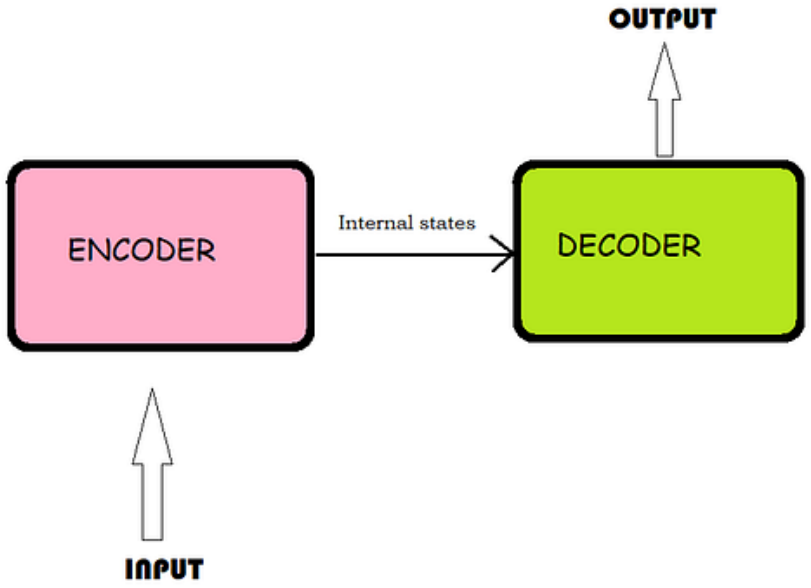


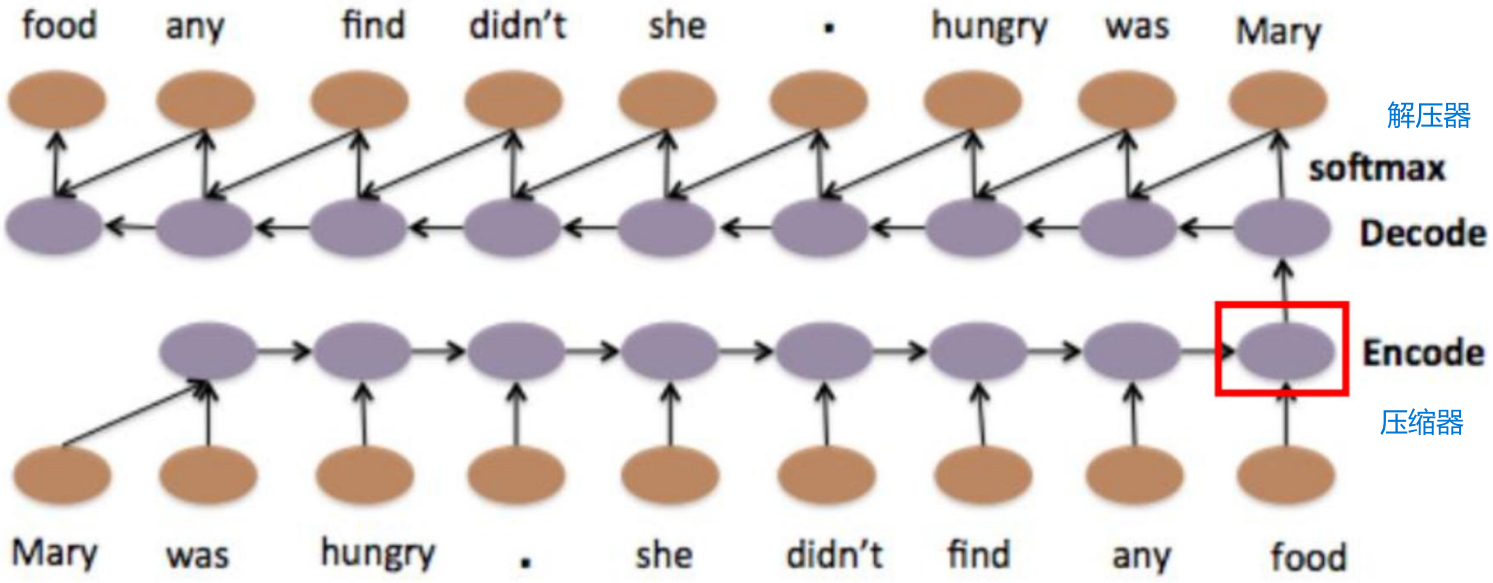


Source: https://goodboychan.github.io/python/deep_learning/tensorflow-keras/2020/12/09/01-RNN-Many-to-many.html

应用 Sequence-to-sequence

- Auto-encoder – Text



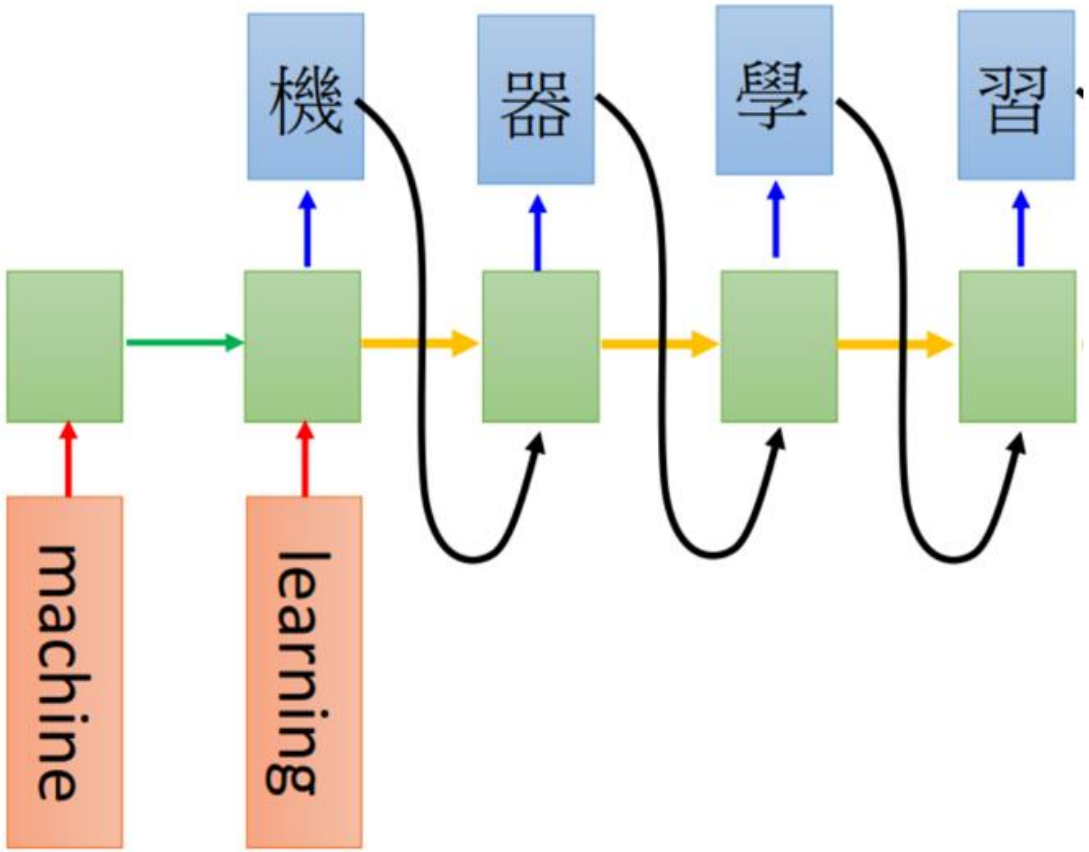


这样就能得到一个高保真的压缩向量

Li, Jiwei, Minh-Thang Luong, and Dan Jurafsky. “A hierarchical neural autoencoder for paragraphs and documents." arXiv preprint arXiv:1506.01057(2015).

应用 Sequence-to-sequence

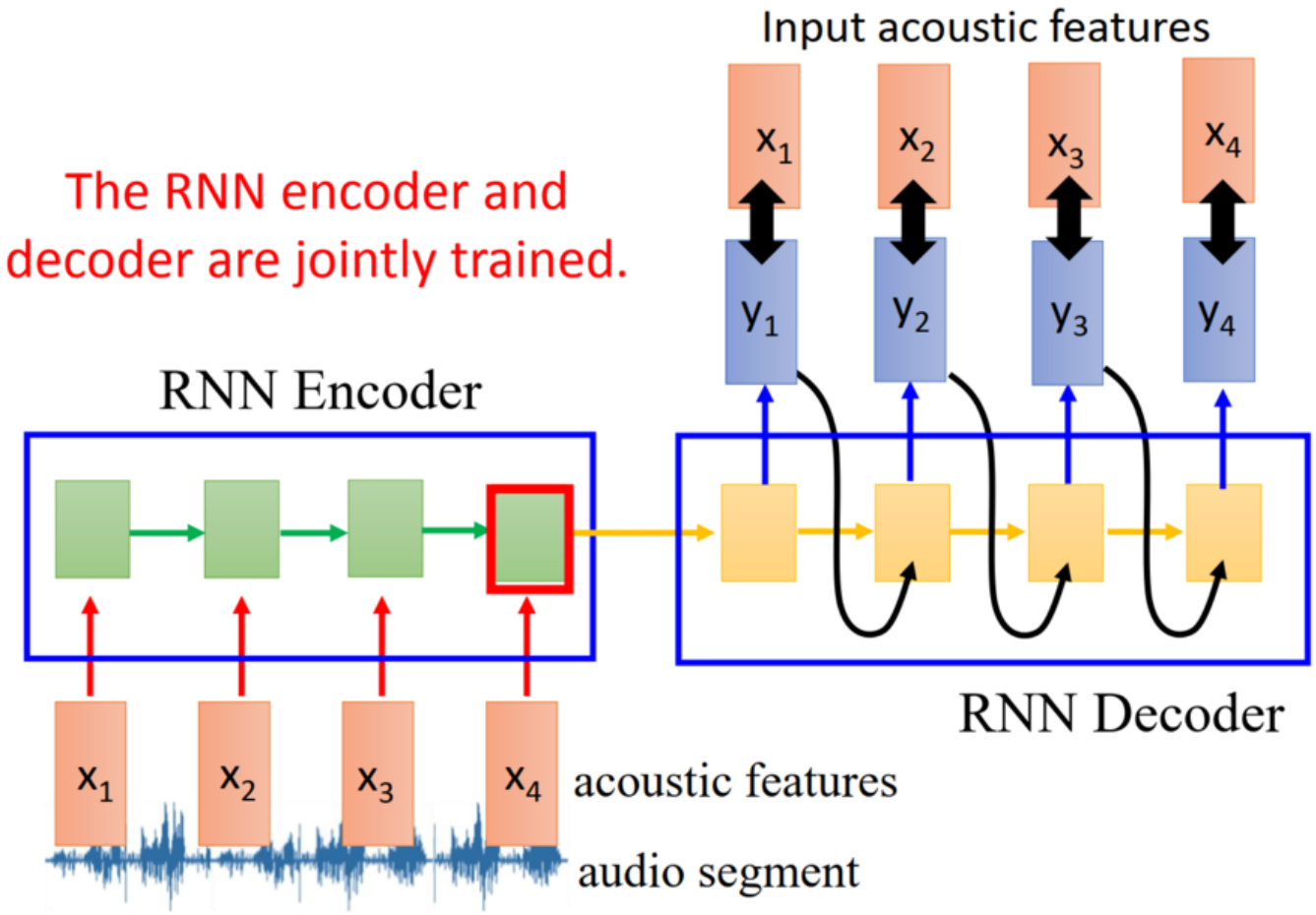
- 输入和输出都是一个vector的序列，而且长度可以不一样
  - E.g., 机器翻译



Source: https://medium.com/analytics-vidhya/machine-translation-encoder-decoder-model-7e4867377161
Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

应用 Sequence-to-sequence

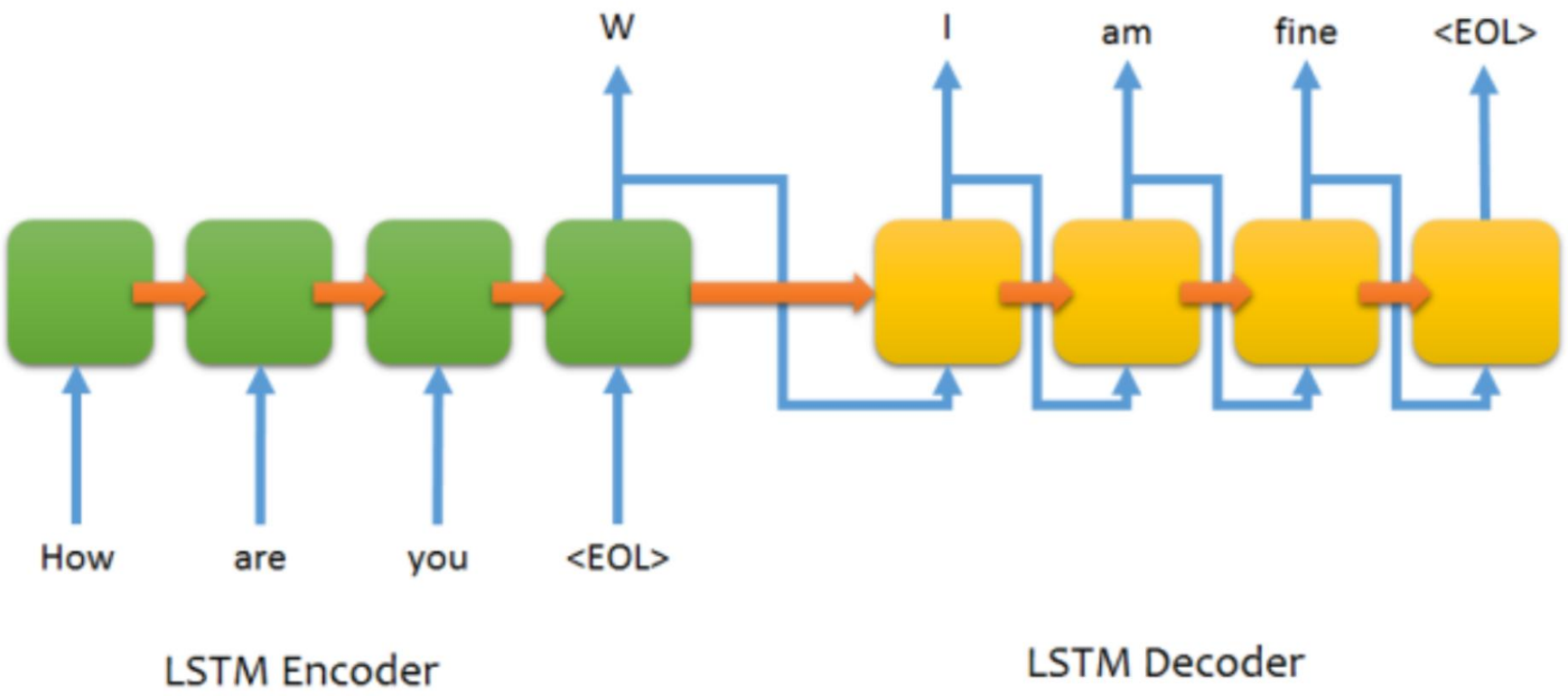
- Auto-encoder – Speech



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php

应用 Sequence-to-sequence

- Chat-bot



Source: https://speech.ee.ntu.edu.tw/~hylee/ml/2020-spring.php